

Backfill Action 与 DRF/Gang/NumaAware 插件

CONTENTS

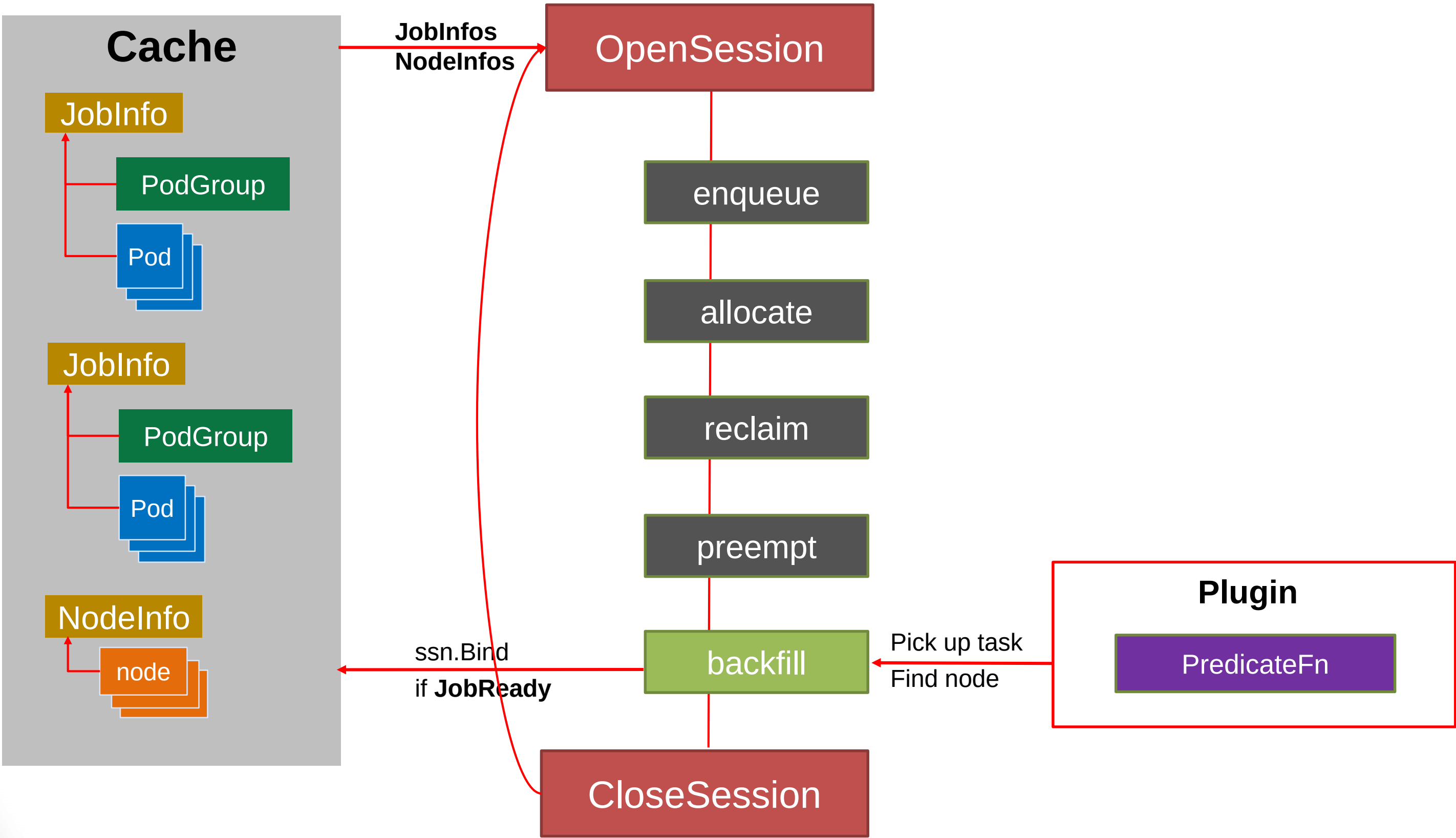
- 01 Backfill action
- 02 DRF 插件
- 03 Gang 插件
- 04 NumaAware 插件

01

Backfill Action



Backfill Action



actions: "enqueue, allocate, **backfill**"

tiers:

- plugins:

- name: priority
- name: gang
- name: conformance

- plugins:

- name: overcommit
- name: drf
- name: predicates
- name: proportion
- name: nodeorder
- name: binpack

Backfill action:

Backfill action是调度流程中的回填步骤，处理待调度Pod列表中没有指明资源申请量的Pod调度，在对单个Pod执行调度动作的时候，遍历所有的节点，只要节点满足了Pod的调度请求，就将Pod调度到这个节点上。

02

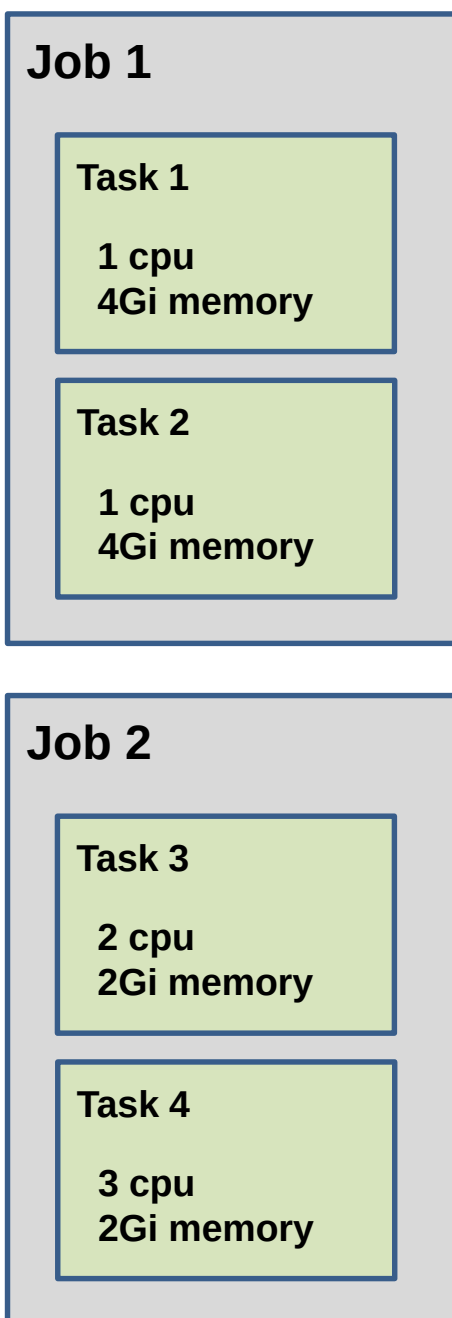
DRF 插件



DRF 插件

Cluster Resources: [10 cpu, 20Gi memory]

Queue 1



Job 1:

Total CPU: $1+1 = 2$

CPU Share = $2/10 = 0.2$

Total memory: $4+4 = 8$

Memory Share = $8/20 = 0.4$

Share = 0.4 [Dominant resource is Memory]

Job 2:

Total CPU: $2+3 = 5$

CPU Share = $5/10 = 0.5$

Total memory: $2+2 = 4$

Memory Share = $4/20 = 0.2$

Share = 0.5 [Dominant resource is CPU]

Job 1 has higher priority than Job 2 as Job 1 share (0.4) is smaller than Job 2 share (0.5).

Dominant Resource Fairness (DRF)

观察每个Job请求的主导资源，并将其作为对集群资源使用的一种度量，根据Job的主导资源，计算Job的share值，在调度的过程中，具有较低share值的Job将具有更高的调度优先级。这样能够满足更多的作业，不会因为一个胖业务，饿死大批小业务。

DRF调度算法能够确保在多种类型资源共存的环境下,尽可能满足分配的公平原则。

使用场景

DRF调度算法优先考虑集群中业务的吞吐量，适用单次AI训练、单次大数据计算以及查询等批处理小业务场景。



03

Gang 插件



Gang 插件

Gang

Gang调度策略是volcano-scheduler的核心调度算法之一，它满足了调度过程中的“All or nothing”的调度需求，避免Pod的任意调度导致集群资源的浪费。具体算法是，观察Job下的Pod已调度数量是否满足了最小运行数量，当Job的最小运行数量得到满足时，为Job下的所有Pod执行调度动作，否则，不执行。

使用场景

基于容器组概念的Gang调度算法十分适合需要多进程协作的场景。AI场景往往包含复杂的流程，Data Ingestion、Data Analysts、Data Splitting、Trainer、Serving、Logging等，需要一组容器进行协同工作，就很适合基于容器组的Gang调度策略。MPI计算框架下的多线程并行计算通信场景，由于需要主从进程协同工作，也非常适合使用Gang调度策略。容器组下的容器高度相关也可能存在资源争抢，整体调度分配，能够有效解决死锁。

Volcano Job example:

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: job-name
spec:
  minAvailable: 3
  schedulerName: volcano
```

PodGroup example:

```
apiVersion: scheduling.volcano.sh/v1beta1
kind: PodGroup
metadata:
  name: podgroup-name
spec:
  minMember: 3
  minResources:
    cpu: 3
    memory: 3Gi
```

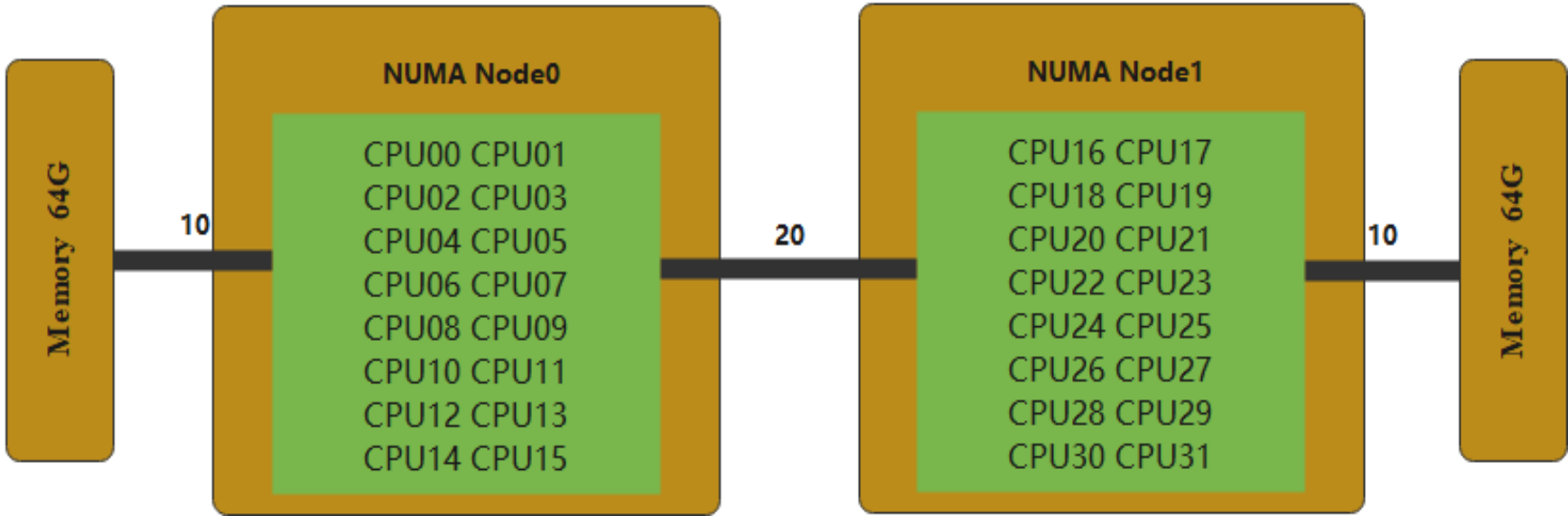


04

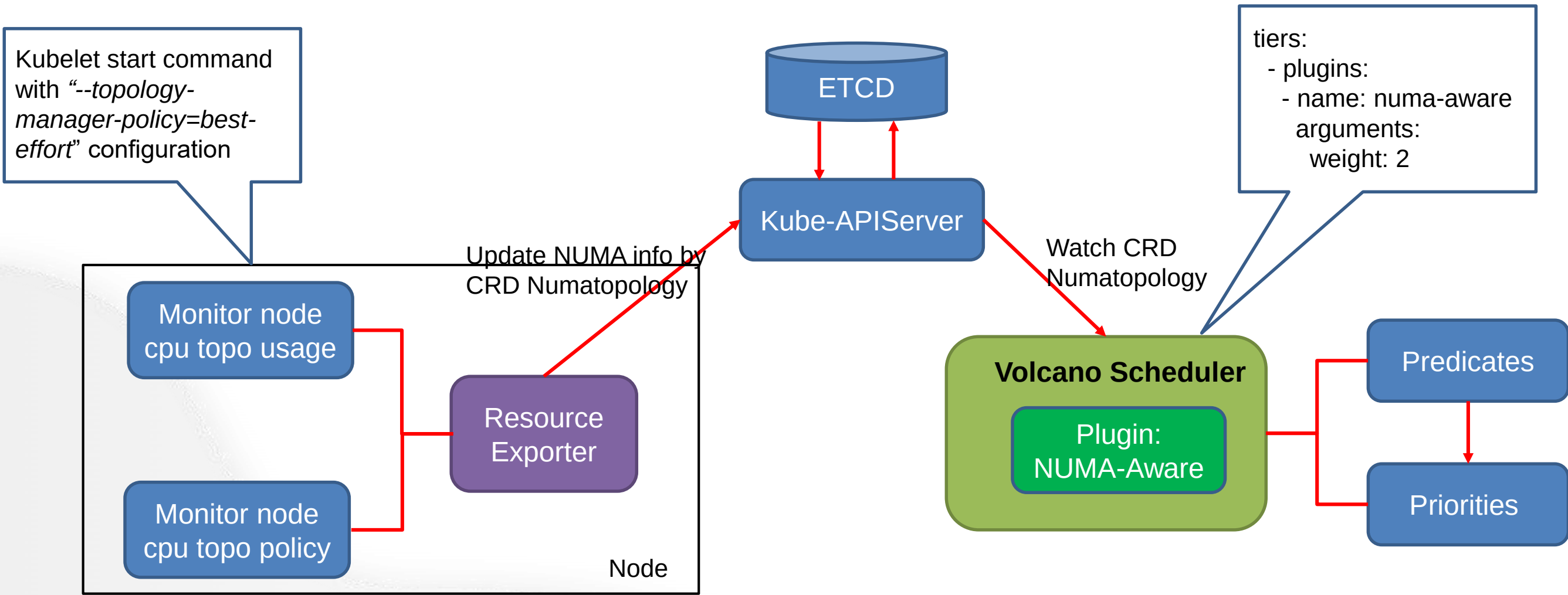
NumaAware 插件



NumaAware 插件



NUMA Architecture



NUMA-Aware scheduling

NumaAware

通过Resource Exporter上报节点CPU拓扑与Pod使用CPU的Numa分布，Volcano感知节点Numa拓扑信息。

使用场景：

Numa-aware的常见场景是那些对cpu参数敏感\调度延迟敏感的计算密集型作业。如科学计算、视频解码、动漫动画渲染、大数据离线处理等具体场景。

NumaAware Example

Volcano Job example

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: vj-test
spec:
  schedulerName: volcano
  minAvailable: 1
  tasks:
    - replicas: 1
      name: "test"
      topologyPolicy: best-effort
      template:
        spec:
          containers:
            - image: alpine
              command: ["/bin/sh", "-c", "sleep 1000"]
  ... ..
```

TF Job example

```
apiVersion: kubeflow.org/v1
kind: TFJob
... ..
spec:
  tfReplicaSpecs:
    PS:
      replicas: 1
      template:
        metadata:
          annotations:
            volcano.sh/numa-topology-policy: "best-effort"
        spec:
          containers:
            - name: tensorflow
              image: gcr.io/your-project/your-image
          ... ..
    Worker:
      replicas: 1
      template:
        metadata:
          annotations:
            volcano.sh/numa-topology-policy: "best-effort"
        spec:
          containers:
            - name: tensorflow
              image: gcr.io/your-project/your-image
          ... ..
```

topologyPolicy support four allocation policies:

- none (default)
- best-effort
- restricted
- single-numa-node

Join Volcano



Website: <https://volcano.sh/en/>



Github: <https://github.com/volcano-sh/volcano>



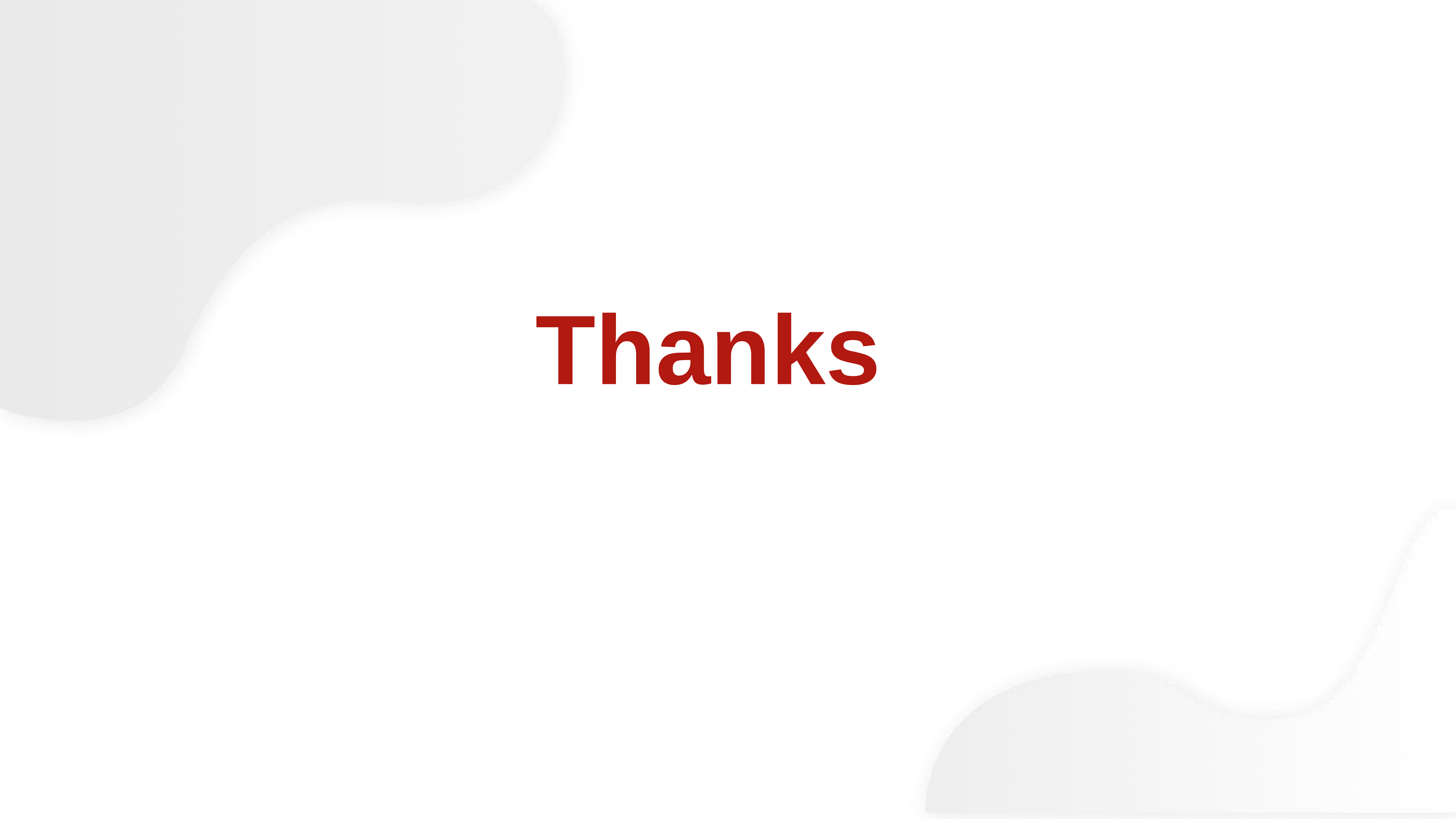
Slack Channel: <https://volcano-sh.slack.com/>



Follow Us



Join Group



Thanks